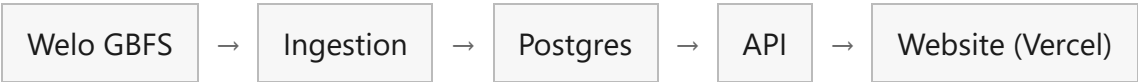


Bonn Mobility — Projektplan

Öffentliche Plattform für Welo in Bonn/Rhein-Sieg: Live zeigen, Historie selbst mitschneiden, daraus Analytics ableiten. Backend zuerst lokal, später Cloud-Kubernetes.

System



Analytics liest Snapshots und schreibt Kennzahlen zurück in Postgres. Container (Ingestion, API, Analytics, Postgres): zuerst Docker Compose auf dem PC, später dieselben Images in Cloud-Kubernetes. Website bleibt auf Vercel. Domain: bonn-mobility.de (IONOS → Vercel DNS).

Dienste

Dienst	Job	Wo
Ingestion	Welo holen, prüfen, Snapshots speichern	Docker → Kubernetes
API	Live, Historie, Analytics an die Website	Docker → Kubernetes
Analytics	Stundenwerte, Health, Trends rechnen	Docker → Kubernetes
Postgres	Stationen, Snapshots, Kennzahlen	Docker → Kubernetes (Volume/Backup)
Website	Karte, Station, Dashboard	Vercel

Anforderungen

Bereich	Inhalt
Produkt	Live-Karte, Stations-Detail, Verfügbarkeitskurven, Health, öffentlich. Browser spricht nie Welo.
Daten	Eigene Minuten-Snapshots. Retention. Kennzahlen aus Historie, nicht aus Fahrten.
Betrieb	Backend containerisiert. Getrennt deployen/skalieren. Healthchecks, CI/CD, Monitoring, Lasttest (k6).

Werkzeuge

Pflicht	Mit dem Cluster	Nur bei Bedarf
Docker, Compose, Postgres-Image, GitHub Actions, Vercel	Kubernetes (z. B. DO/Civo), Ingress, Terraform, Prometheus, Grafana, k6	Helm, Redis, Message Queue, Logging/Tracing, Supabase

Bau-Reihenfolge

1. Postgres + Schema lokal (Compose) — ein Poll speichert Welo-Daten
2. API + Karte — Browser fragt nur /v1
3. Historie + Health — Kurve/Score aus eigenen Snapshots
4. Vier Container lokal — Compose: ein Dienst neu, Rest läuft
5. Cloud-Kubernetes — dieselben Images; Rolling Update der API ohne Ingest-Ausfall
6. Messen + Last — Grafana + k6-Protokoll

Nicht im Kern: Nutzerkonten, Geozonen, echte Fahrten, Fake-ML. Supabase entfällt zugunsten des Postgres-Containers.